

APUNTES DE INICIACIÓN

Unity y Unity Hub

Guía extensa de introducción al motor de videojuegos Unity, la gestión de proyectos con Unity Hub y los conceptos básicos del Editor.

Jorge · DAM 2 · Asignatura: Unity
Curso 2026-2027

Índice

¿Qué es Unity? Introducción al motor	5
1.1. ¿Para qué sirve Unity?	5
1.2. Unity frente a otros motores	5
1.3. Versiones y ciclo de lanzamientos	6
Unity Hub: instalación y gestión	7
2.1. Instalación de Unity Hub	7
2.2. Secciones principales de Unity Hub	7
2.3. Gestión de versiones del Editor	7
2.4. Módulos de plataforma	8
2.5. Gestión de proyectos desde Unity Hub	8
2.6. Unity Cloud, control de versiones y colaboración	8
Crear un proyecto y su estructura de carpetas	9
3.1. Creación de un proyecto nuevo	9
3.2. Estructura de carpetas de un proyecto	9
3.3. La carpeta Assets en detalle	10
La interfaz del Editor de Unity	11
4.1. Paneles principales	11
4.2. La barra de herramientas (Toolbar)	11
4.3. Navegación en la vista Scene	12
4.4. El Inspector	12
GameObjects, Components y Prefabs	13
5.1. ¿Qué es un GameObject?	13
5.2. ¿Qué es un Component?	13
5.3. El componente Transform	13
5.4. Jerarquía: relaciones padre-hijo	13
5.5. Prefabs	14
Materiales, shaders y texturas	15
6.1. Conceptos básicos	15
6.2. Shaders más habituales	15
6.3. Crear y asignar un material	15

6.4. Emission: materiales que “brillan”	15
Iluminación en Unity	17
7.1. Tipos de luz	17
7.2. Propiedades principales de una Light	17
7.3. Ejemplo práctico: un Sol que ilumina una Tierra	17
7.4. Global Illumination (breve introducción)	18
La cámara	19
8.1. Propiedades principales	19
8.2. Ajustar la cámara para encuadrar un objeto	19
8.3. Múltiples cámaras	19
Física básica: Rigidbody y Colliders	20
9.1. Rigidbody	20
9.2. Colliders	20
9.3. Colisiones frente a Triggers	20
Introducción al scripting en C#	21
10.1. MonoBehaviour	21
10.2. Ciclo de vida de un MonoBehaviour	21
10.3. Variables visibles en el Inspector	22
10.4. Buenas prácticas iniciales	22
Entrada del jugador (Input)	23
11.1. Ejemplo con el Input Manager clásico	23
Assets, Asset Store y Package Manager	24
12.1. El Package Manager	24
12.2. La Asset Store	24
12.3. Importar assets externos	24
Build Settings y publicación	25
13.1. Pasos básicos para compilar (build)	25
13.2. Player Settings frecuentes	25
Buenas prácticas y organización del proyecto	26
Glosario y atajos de teclado	27
15.1. Glosario rápido	27

15.2. Atajos de teclado esenciales del Editor 27

Recursos y siguientes pasos 28

Resumen final 28

TEMA 1

¿Qué es Unity? Introducción al motor

Unity es un **motor de videojuegos** (game engine): un conjunto de herramientas de software que permite crear videojuegos y aplicaciones interactivas en 2D, 3D, realidad virtual (VR) y realidad aumentada (AR) sin tener que programar desde cero un renderizador gráfico, un motor de físicas, un sistema de sonido o un sistema de gestión de assets. Fue desarrollado por Unity Technologies y su primera versión se presentó en 2005. Hoy es uno de los motores más usados del mundo, tanto por estudios independientes como por grandes compañías.

Un motor de videojuegos resuelve, de forma reutilizable, los problemas comunes a (casi) cualquier juego: dibujar objetos en pantalla, detectar colisiones, reproducir sonidos, gestionar la entrada del teclado/mando/táctil, cargar y descargar niveles, compilar el proyecto para distintas plataformas, etc. Unity organiza todo esto alrededor de dos ideas centrales que veremos en detalle más adelante: los **GameObjects** (los “objetos” de la escena) y los **Components** (los “comportamientos” que se les añaden).

1.1. ¿Para qué sirve Unity?

Aunque Unity nació como motor de videojuegos, hoy se usa en muchos más campos:

- **Videojuegos:** 2D, 3D, móviles, PC, consolas, navegador.
- **Realidad Virtual y Aumentada:** aplicaciones inmersivas para Meta Quest, HoloLens, ARKit/ARCore.
- **Simulación y formación:** simuladores industriales, médicos, de conducción, entrenamientos.
- **Arquitectura e ingeniería:** visualización arquitectónica interactiva (Unity Reflect / gemelos digitales).
- **Cine y animación:** pre-visualización (previz) y renderizado en tiempo real.
- **Aplicaciones interactivas:** instalaciones artísticas, exposiciones, apps educativas.

1.2. Unity frente a otros motores

En el mercado existen varios motores populares; conviene conocer cómo se posiciona Unity respecto a ellos, ya que en el módulo de Unity de DAM es habitual comparar herramientas:

Motor	Lenguaje principal	Punto fuerte	Uso típico
Unity	C#	Curva de aprendizaje suave, multiplataforma, gran comunidad y Asset Store	Indie, móvil, VR/AR, prototipado rápido
Unreal Engine	C++ / Blueprints	Gráficos fotorrealistas de última generación	AAA, cine, arquitectura de alta gama
Godot	GDScript / C#	100% gratuito y de código abierto, muy ligero	Indies, aprendizaje, 2D
GameMaker	GML	Muy orientado a 2D, rápido para prototipos	Juegos pequeños/medianos 2D

Por qué se estudia Unity en DAM

Unity utiliza **C#** como lenguaje de scripting, el mismo paradigma de programación orientada a objetos que se trabaja en otros módulos del ciclo (Java, programación multiplataforma...). Esto permite aprovechar conocimientos previos de POO, patrones de diseño y estructuras de datos aplicándolos a un ámbito muy visual y motivador: el desarrollo de videojuegos.

1.3. Versiones y ciclo de lanzamientos

Unity publica varias versiones al año, identificadas por año y número (por ejemplo, 2022.3, 2023.1 o, desde el cambio de nomenclatura, **Unity 6** y **Unity 6.x**). Las versiones marcadas como **LTS (Long Term Support)** reciben correcciones de errores durante más tiempo y son las recomendadas para proyectos formativos y de producción, frente a las versiones “Tech Stream”, más recientes pero menos estables. Unity Hub, que veremos en el Tema 2, es precisamente la herramienta que permite tener varias versiones del Editor instaladas a la vez y elegir cuál usa cada proyecto.

TEMA 2

Unity Hub: instalación y gestión

Unity Hub es una aplicación independiente del Editor de Unity que actúa como panel de control: desde ella se instalan y gestionan las distintas versiones del Editor, se crean y abren proyectos, se instalan módulos de plataforma (Android, iOS, WebGL...), se gestiona la licencia y la cuenta de Unity, y se accede a plantillas y tutoriales. Antes de la existencia de Unity Hub, cada versión del Editor se instalaba y actualizaba de forma independiente, lo que era incómodo cuando un desarrollador manejaba varios proyectos con distintas versiones de Unity a la vez.

2.1. Instalación de Unity Hub

- 1 Se descarga el instalador desde la página oficial **unity.com/download**.
- 2 Se ejecuta el instalador (en Windows, un .exe; en macOS, un .dmg; en Linux, un .ApplImage o paquete).
- 3 Al abrir Unity Hub por primera vez, se pide iniciar sesión con una **cuenta de Unity (Unity ID)**, gratuita, necesaria para activar la licencia.
- 4 Se elige el tipo de licencia: **Unity Personal** (gratuita, pensada para estudiantes, aficionados y pequeños estudios con ingresos por debajo de un umbral anual) es la habitual en un contexto educativo.
- 5 Desde la sección *Instalaciones*, se instala al menos una versión del Editor (se recomienda la última LTS disponible).

2.2. Secciones principales de Unity Hub

Sección	Para qué sirve
Proyectos	Lista de proyectos creados o importados; permite abrir, crear, importar y eliminar proyectos de la lista (sin borrar los archivos del disco).
Plantillas	Plantillas de partida: 3D (Built-in), 3D (URP), 2D, VR, Mobile 3D, Aplicación de muestra multiplataforma, etc.
Instalaciones	Gestión de las distintas versiones del Editor instaladas: añadir, actualizar, desinstalar, y añadir <i>módulos</i> (soporte Android, iOS, WebGL, documentación...).
Configuración inicial / Preferencias	Rutas de instalación por defecto, gestor de licencias, tema visual, cuenta y organizaciones.

2.3. Gestión de versiones del Editor

Cada proyecto de Unity queda “atado” a la versión del Editor con la que se creó (se guarda en el archivo `ProjectSettings/ProjectVersion.txt`). Unity Hub permite instalar varias versiones en paralelo, de modo que un proyecto antiguo siga abriéndose con la versión con la que fue creado mientras se empieza otro proyecto nuevo con la versión más reciente. Para cambiar la versión de un proyecto ya existente, se despliega el selector de versión junto al nombre del proyecto en la lista y se elige otra instalada (Unity avisará si el cambio puede requerir una actualización irreversible del proyecto).

2.4. Módulos de plataforma

Al instalar o editar una versión del Editor, Unity Hub ofrece **módulos** adicionales opcionales, necesarios para compilar el proyecto hacia distintas plataformas:

- **Android Build Support** (incluye normalmente el SDK, NDK y OpenJDK necesarios).
- **iOS Build Support** (requiere macOS y Xcode para compilar el paquete final).
- **WebGL Build Support**, para publicar el juego como página web jugable en el navegador.
- **Windows/Mac/Linux Build Support**, para generar ejecutables de escritorio.
- **Documentación y Componentes de idioma** del propio Editor.

Consejo práctico

Si en el futuro se va a compilar el proyecto para móvil (por ejemplo, para las prácticas de este curso), conviene instalar el módulo **Android Build Support** desde el principio: así se evita tener que reinstalar componentes grandes a mitad de curso.

2.5. Gestión de proyectos desde Unity Hub

Desde la pestaña *Proyectos* se puede: crear un proyecto nuevo (*Nuevo proyecto*), añadir a la lista un proyecto que ya existe en el disco (*Añadir > Añadir proyecto desde el disco*), fijar proyectos como favoritos, organizarlos con etiquetas, y *quitar* un proyecto de la lista (esto únicamente desvincula el proyecto de Unity Hub; los archivos permanecen intactos en el disco hasta que se borran manualmente desde el explorador de archivos).

2.6. Unity Cloud, control de versiones y colaboración

Unity Hub también da acceso a servicios en la nube de Unity (Unity Cloud): control de versiones para proyectos (Unity Version Control, antes llamado Plastic SCM), análisis de uso, y servicios de colaboración en equipo. Para un proyecto personal de prácticas como el de esta asignatura, es habitual usar en su lugar **Git** junto con un archivo `.gitignore` específico de Unity, que excluye del control de versiones las carpetas generadas automáticamente (`Library/`, `Temp/`, `obj/...`), ya que Unity las reconstruye solo al volver a abrir el proyecto.

TEMA 3

Crear un proyecto y su estructura de carpetas

3.1. Creación de un proyecto nuevo

Al pulsar *Nuevo proyecto* en Unity Hub, se elige primero una **plantilla** (define el pipeline de renderizado y algunos paquetes iniciales) y después el nombre del proyecto, la ubicación en disco y, opcionalmente, una organización de Unity Cloud.

Plantilla	Descripción
3D (Built-In Render Pipeline)	Plantilla 3D clásica; el pipeline de renderizado “histórico” de Unity, sencillo y compatible con más tutoriales antiguos.
3D (URP)	Universal Render Pipeline: pipeline moderno, más eficiente y multiplataforma (móvil, PC, consolas); recomendado para proyectos nuevos.
3D (HDRP)	High Definition Render Pipeline: gráficos de alta fidelidad, pensado para PC/consola de gama alta; no apto para móvil.
2D (Built-In / URP)	Preconfigurada para juegos 2D: cámara ortográfica, sprites, física 2D.
VR / Mobile	Plantillas orientadas a realidad virtual o a un pipeline optimizado para móviles.

3.2. Estructura de carpetas de un proyecto

Al crear el proyecto, Unity genera una carpeta raíz con el nombre elegido (por ejemplo, My project) que contiene varias subcarpetas. Es importante saber cuáles forman parte del proyecto “real” (y por tanto deben respaldarse y versionarse) y cuáles son cachés regenerables:

Carpeta	Contenido	¿Se versiona?
Assets/	Todos los recursos del proyecto: escenas, scripts, modelos, texturas, materiales, prefabs, audio...	Sí
Packages/	Definición de los paquetes (Package Manager) usados por el proyecto y sus versiones.	Sí
ProjectSettings/	Configuración del proyecto: física, calidad gráfica, input, tags y layers, versión del Editor...	Sí
UserSettings/	Preferencias del usuario local (no del proyecto): layout de ventanas, última escena abierta...	Opcional
Library/	Caché interna generada por Unity al importar los assets (miniaturas, datos compilados...). Se regenera automáticamente.	No
Temp/	Archivos temporales de la sesión de trabajo actual.	No
Logs/	Registros (logs) de compilación y ejecución del Editor.	No

Carpeta	Contenido	¿Se versiona?
obj/ (en scripts)	Archivos intermedios de compilación de C#.	No

¿Por qué no se copia/versiona Library?

La carpeta **Library** puede llegar a pesar varios gigabytes y es totalmente prescindible: contiene únicamente datos que Unity reconstruye automáticamente la primera vez que se abre el proyecto (ese primer arranque tarda algo más, precisamente por tener que reimportar todos los assets). Por eso, al mover, copiar o subir a un repositorio un proyecto de Unity, solo son estrictamente necesarias las carpetas Assets, Packages y ProjectSettings.

3.3. La carpeta Assets en detalle

Dentro de Assets/ el estudiante organiza libremente sus propios recursos. No existe una única estructura obligatoria, pero una organización habitual y recomendable es:

```
Assets/  
  Scenes/      -> Escenas del juego (.unity)  
  Scripts/     -> Todos los scripts en C# (.cs)  
  Prefabs/     -> Prefabs reutilizables  
  Materials/   -> Materiales (.mat)  
  Textures/    -> Imágenes y texturas  
  Models/      -> Modelos 3D importados (.fbx, .obj...)  
  Audio/       -> Música y efectos de sonido  
  UI/          -> Sprites y prefabs de interfaz  
  Settings/    -> Perfiles de URP/HDRP, Input Actions...
```

TEMA 4

La interfaz del Editor de Unity

Al abrir un proyecto desde Unity Hub se inicia el **Editor**, la aplicación principal donde se diseña el juego. Por defecto, el Editor se organiza en varias ventanas o *paneles* acoplables, que juntas forman un **Layout** (disposición). Se puede cambiar entre layouts predefinidos, o crear uno propio, desde el menú desplegable *Layout* situado en la esquina superior derecha.

4.1. Paneles principales

Panel	Función
Scene	Vista 3D/2D editable de la escena: aquí se colocan, mueven y rotan los objetos con el ratón. No es lo que ve el jugador, sino la vista de trabajo del diseñador.
Game	Vista previa de lo que realmente verá el jugador a través de la cámara activa; se actualiza al pulsar Play.
Hierarchy	Lista en árbol de todos los GameObjects presentes en la escena actual, mostrando las relaciones padre-hijo.
Inspector	Muestra y permite editar todas las propiedades y componentes del objeto seleccionado (en la Hierarchy o en el Project).
Project	Explorador de todos los archivos (assets) del proyecto, organizados en carpetas, tal y como están guardados en disco dentro de Assets/.
Console	Muestra mensajes informativos, advertencias (warnings) y errores, tanto del Editor como de los scripts (Debug.Log, por ejemplo).

4.2. La barra de herramientas (Toolbar)

En la parte superior del Editor se sitúa la Toolbar, con las herramientas de transformación y los controles de reproducción:

- **Herramientas de transformación:** Mano/Pan (Q), Mover (W), Rotar (E), Escalar (R), Rect Transform (T) y la herramienta combinada Move+Rotate+Scale (Y).
- **Pivote/Centro y Local/Global:** cambian el punto de referencia y los ejes usados al transformar objetos.
- **Play / Pause / Step:** inician, pausan y avanzan fotograma a fotograma la simulación del juego dentro del propio Editor, sin necesidad de compilarlo.
- **Layers y Layout:** controlan qué capas se ven en la Scene view y la disposición de paneles.

Importante: modo Play

Mientras el Editor está en modo **Play** (fondo de la interfaz teñido de azul), cualquier cambio que se haga sobre los objetos de la escena **no se guarda**: al salir de Play, la escena vuelve al estado anterior. Para hacer cambios permanentes (añadir objetos, ajustar posiciones...) hay que hacerlo con el juego detenido.

4.3. Navegación en la vista Scene

Acción	Atajo (ratón/teclado)
Orbitar la cámara alrededor de un punto	Alt + clic izquierdo + arrastrar
Desplazar la vista (pan)	Clic central (rueda) + arrastrar, o Q + clic izquierdo
Zoom	Rueda del ratón, o Alt + clic derecho + arrastrar
Volar en primera persona por la escena	Clic derecho + WASD (mantener pulsado)
Centrar la vista en el objeto seleccionado	Tecla F

4.4. El Inspector

El Inspector es, probablemente, el panel en el que más tiempo se pasa: muestra el nombre del objeto seleccionado, su etiqueta (Tag) y capa (Layer), y la lista de **componentes** añadidos, empezando siempre por el **Transform** (posición, rotación y escala), ya que todo GameObject lo tiene de forma obligatoria. Desde el botón *Add Component*, situado al final del Inspector, se añaden nuevos componentes (luces, físicas, scripts propios, etc.).

TEMA 5

GameObjects, Components y Prefabs

5.1. ¿Qué es un GameObject?

Un **GameObject** es el elemento básico de cualquier escena de Unity: representa “cualquier cosa” que pueda existir en el juego (un personaje, una cámara, una luz, un menú de interfaz, o incluso un objeto vacío usado solo para organizar la jerarquía). Por sí mismo, un GameObject recién creado no hace nada: no tiene forma, ni física, ni comportamiento. Todo eso se lo dan los **componentes** que se le añaden.

5.2. ¿Qué es un Component?

Un **Component** (componente) es un bloque de funcionalidad que se “engancha” a un GameObject. Unity trae de fábrica decenas de componentes (Mesh Renderer para dibujar una malla 3D, Rigidbody para física, Collider para detectar colisiones, Light para emitir luz, Camera para renderizar la escena, AudioSource para reproducir sonido...), y además permite crear **componentes propios en C#**: un script que hereda de MonoBehaviour se comporta exactamente igual que cualquier otro componente y se añade de la misma manera.

Este diseño se conoce como **composición sobre herencia** (o modelo Entity-Component): en lugar de definir una clase “Jugador” que hereda de “Personaje” que hereda de “GameObject” (herencia profunda y rígida), un jugador es simplemente un GameObject al que se le van añadiendo componentes independientes (movimiento, salud, inventario, sonido...), cada uno responsable de una única responsabilidad.

5.3. El componente Transform

Todo GameObject tiene siempre, de forma obligatoria y no eliminable, un componente **Transform**, con tres propiedades vectoriales (X, Y, Z):

- **Position**: posición en el espacio (mundial o relativa a su padre).
- **Rotation**: orientación, expresada en ángulos de Euler en el Inspector (internamente Unity usa cuaterniones para evitar el problema del *gimbal lock*).
- **Scale**: escala en cada eje respecto a su tamaño original.

5.4. Jerarquía: relaciones padre-hijo

En el panel Hierarchy, un GameObject puede arrastrarse dentro de otro para convertirlo en su **hijo**. El Transform de un hijo se expresa entonces de forma *relativa* a su padre: si el padre se mueve, rota o escala, todos sus hijos se mueven, rotan y escalan con él, conservando su posición relativa. Esto es muy útil para agrupar piezas de un mismo objeto compuesto (por ejemplo, las ruedas de un coche como hijas del chasis) o para organizar la escena en carpetas lógicas (un objeto vacío llamado “-- Luces --” que agrupa todas las luces, por ejemplo).

Objetos “independientes” en la jerarquía

Cuando dos objetos deben moverse de forma **independiente** el uno del otro (por ejemplo, un Sol y una Tierra que orbita a su alrededor con su propio script, o simplemente dos objetos sueltos de la escena), deben colocarse al mismo nivel en la Hierarchy, sin que uno sea hijo del otro. Si, en cambio, se quisiera que la Tierra *siempre* se desplazase junto al Sol pero pudiera rotar por su cuenta, sí interesaría anidarla como hija.

5.5. Prefabs

Un **Prefab** es una plantilla reutilizable de GameObject, guardada como asset dentro del proyecto (extensión .prefab). Permite crear una “receta” de un objeto (por ejemplo, un enemigo, una bala o una moneda) con todos sus componentes ya configurados, e **instanciarla** tantas veces como se necesite, tanto manualmente arrastrándola a la escena como por código mediante `Instantiate()`. Si después se modifica el Prefab original, el cambio se propaga automáticamente a todas sus instancias (salvo las propiedades que se hayan sobrescrito manualmente en una instancia concreta, marcadas en negrita en el Inspector).

TEMA 6

Materiales, shaders y texturas

6.1. Conceptos básicos

Para que una malla 3D (o un sprite 2D) se vea de una determinada manera en pantalla, Unity necesita un **Material**. Un material combina un **Shader** (el programa que calcula cómo se pinta cada píxel: cómo reacciona a la luz, si es transparente, metálico, etc.) con un conjunto de parámetros y **texturas** (imágenes que aportan color, relieve, brillo...). Un mismo Shader puede reutilizarse en decenas de materiales distintos, simplemente cambiando sus texturas y valores.

6.2. Shaders más habituales

Shader	Pipeline	Uso
Standard	Built-In Render Pipeline	Shader PBR (físicamente realista) genérico, todo tipo de superficies.
Universal Render Pipeline/Lit	URP	Equivalente a Standard pero optimizado para URP; el más usado en proyectos nuevos.
Universal Render Pipeline/Unlit	URP	No reacciona a la luz; útil para interfaces, efectos o estilos "flat".
Sprites/Default	2D	Shader por defecto para sprites en proyectos 2D.

6.3. Crear y asignar un material

- 1 En el panel Project, botón derecho sobre una carpeta > *Create > Material*.
- 2 Se le da un nombre descriptivo (por ejemplo, "M_Sol" o "Mat_Suelo").
- 3 En el Inspector del material se elige el Shader deseado y se ajustan sus propiedades (Color/Albedo, Metallic, Smoothness, texturas...).
- 4 El material se arrastra directamente sobre el objeto en la vista Scene, o sobre el campo *Materials* del componente *Mesh Renderer* en el Inspector.

6.4. Emission: materiales que "brillan"

La mayoría de shaders PBR (Standard, URP/Lit) incluyen una sección **Emission**. Al activarla y asignarle un color (y una intensidad, mediante el control HDR de color), la superficie del objeto se dibuja como si emitiera luz propia. Es importante matizar que, por defecto, **Emission solo afecta al aspecto visual del objeto**: no ilumina automáticamente a los objetos de alrededor salvo que, además, se genere Global Illumination (horneada o en tiempo real) a partir de esa emisión, o se combine con un componente *Light* real, como se explica en el siguiente tema. Por eso, para representar un Sol creíble, lo habitual es combinar un material con Emission (para que se vea

brillante) con un componente Light de tipo Point o Directional (para que realmente ilumine la escena).

TEMA 7

Iluminación en Unity

El componente **Light** permite que un `GameObject` emita luz sobre la escena. Al crear un proyecto 3D nuevo, Unity añade automáticamente una *Directional Light* a la escena, simulando la luz del sol.

7.1. Tipos de luz

Tipo	Comportamiento	Ejemplo típico
Directional Light	Ilumina toda la escena desde una dirección única, como si la fuente estuviera infinitamente lejos (los rayos llegan paralelos). Su posición no importa, solo su rotación.	El sol o la luna vistos desde la superficie terrestre
Point Light	Emite luz en todas direcciones por igual desde un punto, con un alcance (Range) limitado.	Una bombilla, una vela, o un sol "local" dentro de la escena
Spot Light	Emite luz en forma de cono, con un ángulo (Spot Angle) y un alcance definidos.	Un foco, una linterna, los faros de un vehículo
Area Light	Emite luz desde la superficie de un rectángulo o disco; solo se usa en iluminación horneada (no en tiempo real).	Un panel de luz de estudio fotográfico

7.2. Propiedades principales de una Light

- **Intensity**: brillo de la luz; a mayor valor, más intensamente ilumina (y más "quema" los blancos).
- **Range** (Point/Spot): distancia máxima hasta la que llega la luz.
- **Color**: color de la luz emitida (por ejemplo, un tono cálido amarillo-anaranjado para un sol).
- **Shadow Type**: si la luz proyecta sombras (Hard/Soft Shadows) o no (No Shadows); las sombras tienen coste de rendimiento.
- **Mode**: Realtime (se recalcula cada fotograma), Baked (precalculada, muy eficiente pero estática) o Mixed (combinación de ambas).

7.3. Ejemplo práctico: un Sol que ilumina una Tierra

Un ejercicio clásico de iniciación consiste en representar un pequeño sistema Sol-Tierra. Los pasos básicos serían:

- 1 Crear una esfera para el Sol (`GameObject > 3D Object > Sphere`) y otra para la Tierra, **sin anidar una dentro de la otra** en la Hierarchy, y separadas en el eje X del Transform (por ejemplo, Sol en (0,0,0) y Tierra en (8,0,0)).
- 2 Crear un material con Emission en tono amarillo/naranja y asignárselo al Sol, para que se vea "incandescente".
- 3 Añadir al Sol un componente *Light* de tipo **Point Light**, con Intensity e Range suficientes para alcanzar a la Tierra.
- 4 Opcionalmente, reducir la intensidad (o eliminar) la *Directional Light* por defecto de la escena, para que se aprecie con claridad que la única fuente de luz relevante es el Sol creado.

- 5 Añadir un script sencillo a la Tierra que la haga rotar sobre sí misma (`transform.Rotate`) y, si se desea, orbitar alrededor del Sol.

7.4. Global Illumination (breve introducción)

Más allá de la luz directa, Unity puede calcular **iluminación global** (cómo la luz rebota entre superficies). Existen dos aproximaciones: la iluminación **horneada** (Baked Lightmapping), que precalcula el resultado y lo guarda como textura (rápida en tiempo real, pero solo válida para geometría estática), y la iluminación en tiempo real, más costosa pero válida para escenas dinámicas. En proyectos de iniciación no suele ser necesario profundizar en este apartado, pero conviene saber que existe.

TEMA 8

La cámara

El componente **Camera** define qué parte de la escena se renderiza hacia la pantalla (panel Game). Todo proyecto 3D nuevo incluye una *Main Camera* por defecto.

8.1. Propiedades principales

Propiedad	Descripción
Projection	Perspective (perspectiva, con fuga hacia el horizonte, habitual en 3D) u Orthographic (sin perspectiva, objetos del mismo tamaño sea cual sea su distancia; habitual en 2D).
Field of View (FOV)	Ángulo de visión de la cámara en modo Perspective; un valor mayor da sensación de gran angular, uno menor de teleobjetivo/zoom.
Clipping Planes (Near/Far)	Distancias mínima y máxima que la cámara renderiza; lo que quede fuera de ese rango no se dibuja (optimiza rendimiento).
Viewport Rect	Qué zona de la pantalla ocupa esta cámara (útil para pantalla partida).
Clear Flags / Background	Qué se dibuja donde no hay geometría: un color sólido (Skybox por defecto), o se deja ver la cámara anterior.

8.2. Ajustar la cámara para encuadrar un objeto

Para colocar la cámara “a mano” apuntando a un objeto recién creado (por ejemplo, la esfera del ejercicio inicial): se selecciona el objeto, se sitúa el cursor sobre la vista Scene y se pulsa **Shift + F** (o se arrastra la cámara con Alt+clic para orbitar alrededor del objeto con la tecla F). Una vez la vista Scene muestra el encuadre deseado, con la Main Camera seleccionada se usa el atajo *GameObject > Align With View* (o Ctrl/Cmd+Shift+F) para que la cámara adopte exactamente esa posición y rotación. También puede ajustarse manualmente el Transform de la cámara (Position/Rotation) desde el Inspector para obtener valores exactos y repetibles.

8.3. Múltiples cámaras

Una escena puede tener varias cámaras (por ejemplo, una cámara cenital de mapa y otra en tercera persona). Cada cámara tiene una propiedad **Depth** que determina el orden de renderizado, y puede activarse o desactivarse por script cambiando la propiedad `enabled` del componente, o activando/desactivando el propio *GameObject*.

TEMA 9

Física básica: Rigidbody y Colliders

9.1. Rigidbody

El componente **Rigidbody** hace que un objeto quede sujeto al motor de físicas de Unity: cae por efecto de la gravedad, responde a fuerzas e impulsos, y puede chocar contra otros objetos. Sin un Rigidbody, un GameObject es puramente *cinemático* (solo se mueve si un script cambia su Transform manualmente) y no interactúa con la física de forma realista.

9.2. Colliders

Un **Collider** define la forma que usa el motor de físicas para detectar colisiones; no tiene por qué coincidir exactamente con la malla visual (a menudo se usa una forma simplificada por rendimiento).

Collider	Forma	Coste
Box Collider	Caja alineada con los ejes del objeto	Muy bajo
Sphere Collider	Esfera	Muy bajo
Capsule Collider	Cápsula (cilindro con extremos redondeados)	Bajo
Mesh Collider	Forma exacta de una malla 3D	Alto (usar con moderación)

9.3. Colisiones frente a Triggers

Si la casilla **Is Trigger** de un Collider está desactivada, el objeto se comporta como sólido: bloquea el paso de otros colliders y genera los eventos `OnCollisionEnter`, `OnCollisionStay` y `OnCollisionExit`. Si está **activada**, el objeto deja de ser sólido (otros objetos lo atraviesan) pero genera los eventos `OnTriggerEnter`, `OnTriggerStay` y `OnTriggerExit`, muy usados para detectar “zonas” (recoger un objeto, entrar en un área, activar un checkpoint...).

TEMA 10

Introducción al scripting en C#

10.1. MonoBehaviour

Un script de Unity es una clase de C# que hereda de MonoBehaviour. Al guardarlo y volver al Editor, Unity lo compila automáticamente y permite añadirlo a cualquier GameObject como un componente más, desde *Add Component* o arrastrándolo directamente sobre el objeto en la Hierarchy.

```
using UnityEngine;

public class Rotador : MonoBehaviour
{
    [SerializeField] private float velocidad = 20f;

    void Start()
    {
        Debug.Log("El objeto " + gameObject.name + " ha comenzado.");
    }

    void Update()
    {
        transform.Rotate(Vector3.up, velocidad * Time.deltaTime);
    }
}
```

10.2. Ciclo de vida de un MonoBehaviour

Unity llama automáticamente, en un orden determinado, a ciertos métodos “mágicos” si existen en el script (no hace falta declararlos todos, solo los que se necesiten):

Método	Cuándo se ejecuta
Awake()	Al cargar el objeto, antes que cualquier Start (útil para inicializar referencias propias).
Start()	Una sola vez, justo antes del primer Update, y solo si el script está habilitado.
Update()	Una vez por fotograma; aquí va la lógica que depende del tiempo real transcurrido (usar Time.deltaTime).
FixedUpdate()	A intervalos fijos, independientes de la tasa de fotogramas; aquí va la lógica de físicas (fuerzas sobre un Rigidbody).
OnCollisionEnter(Collision c)	Al iniciarse una colisión física con otro Collider no-trigger.
OnTriggerEnter(Collider c)	Al entrar en la zona de un Collider marcado como Trigger.
OnDestroy()	Justo antes de que el objeto sea destruido.

10.3. Variables visibles en el Inspector

Cualquier campo **público** de un script (o privado marcado con el atributo `[SerializeField]`, la forma recomendada) aparece automáticamente como un campo editable en el Inspector cuando el script está añadido a un objeto. Esto permite ajustar valores (velocidades, referencias a otros objetos, prefabs...) sin tener que tocar el código, algo muy valioso para iterar rápido durante el diseño del juego.

10.4. Buenas prácticas iniciales

- Un script, una responsabilidad: evitar clases “cajón de sastre” que lo controlen todo.
- Usar `[SerializeField] private` en lugar de campos públicos sueltos, para mantener el encapsulamiento y a la vez poder editar el valor desde el Inspector.
- Cachear referencias costosas de obtener (por ejemplo, `GetComponent()`) en `Awake()` o `Start()`, no en `Update()`.
- Nombrar las clases y los archivos igual (Unity lo exige cuando el nombre del archivo no coincide con la clase pública que contiene).

TEMA 11

Entrada del jugador (Input)

Unity ofrece dos sistemas para leer la entrada del jugador (teclado, ratón, mando, pantalla táctil): el **Input Manager** clásico (más simple, basado en la clase estática `Input`) y el **Input System** moderno (paquete instalable desde el Package Manager, basado en eventos y más flexible para mandos, varios jugadores o remapeo de controles).

11.1. Ejemplo con el Input Manager clásico

```
void Update()
{
    float horizontal = Input.GetAxis("Horizontal"); // A/D o flechas
    float vertical    = Input.GetAxis("Vertical");   // W/S o flechas

    Vector3 direccion = new Vector3(horizontal, 0f, vertical);
    transform.Translate(direccion * velocidad * Time.deltaTime);

    if (Input.GetKeyDown(KeyCode.Space))
    {
        Debug.Log("Salto!");
    }
}
```

Para proyectos nuevos, Unity recomienda el **Input System**, instalable desde *Window > Package Manager*, que se basa en “Input Actions” configurables visualmente y desacopladas de una tecla concreta (por ejemplo, la acción “Mover” puede recibir entrada del teclado, de un mando o de un joystick táctil sin cambiar el código).

TEMA 12

Assets, Asset Store y Package Manager

12.1. El Package Manager

Desde *Window > Package Manager* se instalan, actualizan y eliminan **paquetes**: piezas de funcionalidad oficial o de terceros que se integran en el proyecto (el propio Input System, URP, Cinemachine para cámaras avanzadas, ProBuilder para modelado rápido, TextMeshPro para texto avanzado...). Los paquetes instalados quedan reflejados en la carpeta *Packages/* del proyecto.

12.2. La Asset Store

La **Unity Asset Store** es una tienda (con contenido gratuito y de pago) de modelos 3D, texturas, sonidos, plantillas completas de juego, herramientas de Editor y otros recursos creados por la comunidad y por Unity Technologies. Es una fuente habitual de assets para prototipar rápido sin tener que crear todo el arte desde cero.

12.3. Importar assets externos

Cualquier archivo (modelo 3D, imagen, audio...) copiado dentro de la carpeta *Assets* del proyecto (por ejemplo, desde el explorador de archivos del sistema operativo, o arrastrándolo directamente al panel *Project*) es detectado e **importado** automáticamente por Unity, que genera los metadatos necesarios (archivo *.meta* asociado) y lo deja listo para usarse en la escena.

TEMA 13

Build Settings y publicación

Cuando el proyecto (o una primera versión jugable) está listo, se genera un ejecutable independiente del Editor desde *File > Build Settings* (o *Build Profiles* en versiones recientes de Unity).

13.1. Pasos básicos para compilar (build)

- 1 Abrir *File > Build Settings*.
- 2 Pulsar *Add Open Scenes* para añadir la escena actual a la lista de escenas incluidas en el build (el orden de la lista importa: la escena en la posición 0 es la que se carga al iniciar la aplicación).
- 3 Elegir la **plataforma** de destino (PC/Mac/Linux, Android, WebGL...); si el módulo correspondiente no está instalado, Unity Hub ofrece instalarlo en ese momento.
- 4 Ajustar, si procede, las *Player Settings* (icono de la aplicación, nombre del paquete, orientación de pantalla, resolución...).
- 5 Pulsar *Build* (genera solo el ejecutable) o *Build and Run* (lo genera y lo ejecuta de inmediato).

13.2. Player Settings frecuentes

- **Company Name / Product Name:** identifican la aplicación generada.
- **Icon:** icono de la aplicación compilada.
- **Resolution and Presentation:** resolución por defecto, modo ventana/pantalla completa, orientación (en móvil).
- **Other Settings > Scripting Backend:** Mono (compilación más rápida) o IL2CPP (mejor rendimiento final, obligatorio en algunas plataformas como iOS).

TEMA 14

Buenas prácticas y organización del proyecto

- **Nomenclatura consistente:** prefijos por tipo de asset (M_ para materiales, T_ para texturas, SFX_ para sonidos...) facilitan localizar recursos en proyectos grandes.
- **Una escena de pruebas** separada de las escenas “reales” del juego, para experimentar sin riesgo.
- **Guardar con frecuencia** (Ctrl/Cmd+S guarda la escena; Ctrl/Cmd+Shift+S guarda el proyecto entero) y hacer copias de seguridad o usar control de versiones.
- **Prefabs para todo lo repetido:** si un objeto se va a instanciar más de una vez, casi siempre conviene convertirlo en Prefab desde el principio.
- **Evitar lógica pesada dentro de Update()** cuando pueda resolverse con eventos o corutinas, por rendimiento.
- **Control de versiones con Git:** usar el .gitignore oficial de Unity (excluye Library/, Temp/, Logs/, Obj/...) y activar, en Project Settings > Editor, el modo *Visible Meta Files* y *Force Text* para que los archivos de Unity sean legibles y combinables (mergeables) por Git.

TEMA 15

Glosario y atajos de teclado

15.1. Glosario rápido

Término	Significado
Asset	Cualquier recurso del proyecto: script, escena, textura, modelo, audio, material...
Scene	Un "nivel" o pantalla del juego; contiene una jerarquía de GameObjects.
GameObject	Elemento básico de una escena; contenedor de componentes.
Component	Bloque de funcionalidad añadido a un GameObject.
Prefab	Plantilla reutilizable de GameObject, guardada como asset.
Script	Componente propio escrito en C#, heredado de MonoBehaviour.
Inspector	Panel que muestra y permite editar las propiedades del objeto seleccionado.
Build	Versión compilada y ejecutable del proyecto, generada para una plataforma concreta.
Render Pipeline	Conjunto de algoritmos que determinan cómo se dibuja (renderiza) la escena (Built-In, URP, HDRP).
Package Manager	Herramienta para instalar/actualizar paquetes oficiales o de terceros.

15.2. Atajos de teclado esenciales del Editor

Atajo	Acción
Q / W / E / R	Herramientas Mano / Mover / Rotar / Escalar
F	Centrar la vista Scene en el objeto seleccionado
Ctrl/Cmd + S	Guardar la escena actual
Ctrl/Cmd + Shift + S	Guardar el proyecto
Ctrl/Cmd + D	Duplicar el objeto seleccionado
Ctrl/Cmd + Z / Y	Deshacer / Rehacer
Ctrl/Cmd + P	Entrar/salir del modo Play
Alt + clic izq. + arrastrar	Orbitar la cámara de la vista Scene

TEMA 16

Recursos y siguientes pasos

Para seguir avanzando más allá de estos apuntes, estas son las fuentes de referencia oficiales más recomendables:

- **Documentación oficial** (Manual y Scripting API): docs.unity3d.com
- **Unity Learn**: cursos y proyectos guiados oficiales, gratuitos, de nivel principiante a avanzado: learn.unity.com
- **Unity Discussions / Foros**: comunidad oficial de preguntas y respuestas.
- **Documentación del Input System, URP, Cinemachine...**: cada paquete instalado desde el Package Manager incluye su propia documentación, accesible desde el propio gestor de paquetes.

Resumen final

Con lo visto en esta guía ya es posible: instalar y mantener Unity Hub y el Editor, crear y organizar un proyecto nuevo, moverse con soltura por la interfaz, crear y relacionar GameObjects con sus componentes, dar apariencia a los objetos con materiales, iluminar una escena de forma realista, colocar la cámara, dotar de física básica a los objetos, escribir los primeros scripts en C# y, finalmente, generar un build ejecutable del proyecto. Estas son exactamente las bases sobre las que se construyen el resto de prácticas de la asignatura de Unity de este curso.